

DC-FOUNDATION-01 — Complete Study Guide

Understand every command, concept, and screenshot in the lab, from zero

Redji Jean Baptiste (Toussaint)

Companion to the DC-FOUNDATION-01 Lab Report

How to use this. This assumes you executed the lab by following the manual line-by-line without necessarily understanding it. It goes back over everything: what each piece does, why it is there, what the output means, and where it bites. Every part ends with a **Skill Check** — cover the answers and test yourself. When the Master Exam (§12) is effortless, the lab is yours.

Contents

1	Mental model: what you actually built	2
2	Foundations you need first	2
2.1	The shell / terminal	2
2.2	Root, sudo, least privilege	2
2.3	SSH in one line	2
3	Imaging and first boot	2
4	Baseline capture	3
5	Updating the system	4
6	Networking deep dive	4
6.1	IP addresses and CIDR	4
6.2	Gateway and DNS	4
6.3	DHCP vs static; NetworkManager	4
6.4	APIPA	4
7	Users and permissions	5
8	SSH keys and hardening — the heart of the lab	5
8.1	Asymmetric crypto	5
8.2	Distributing the key	5
8.3	Host keys	5
8.4	The include-ordering trap (D3)	5
8.5	Proving it: sshd -T and the negative test	6
9	Firewall, fail2ban, patching, time	7
10	Git, GitHub, and Docker	7
11	The nine defects as teaching cases	8

12 Master skill-check exam**9**

1 Mental model: what you actually built

You turned a bare Raspberry Pi into a **hardened headless Linux server**. Four words, the whole lab:

- **Server** — an always-on computer that provides services over a network rather than being sat in front of.
- **Headless** — no screen or keyboard; reached entirely over SSH. Nothing was ever plugged into the Pi.
- **Hardened** — deliberately locked down: key-only logins, default-deny firewall, auto-patching, brute-force protection.
- **Linux** — Raspberry Pi OS, i.e. Debian, on the Pi's ARM chip.

2 Foundations you need first

2.1 The shell / terminal

The prompt `admin@dc-node-01:~ $` is information: `admin` = the user; `dc-node-01` = the machine; `~` = your home directory; `$` = ordinary user (a `#` would mean root). This is how you tell the Pi (bash, `admin@dc-node-01:~ $`) from your laptop (PowerShell, `PS C:\Users\jredj>`) — several errors came from running a command on the wrong one.

2.2 Root, sudo, least privilege

Root can do anything, which is dangerous. You log in as limited `admin` and borrow root power per-command with `sudo`. This is **least privilege** — a theme of the whole lab.

2.3 SSH in one line

SSH gives you an encrypted remote shell on port 22; everything you type travels encrypted.

Skill Check

1. What does `~` mean, and what would `#` instead of `$` tell you?
2. Why is `admin+sudo` safer than logging in as root?
3. What does SSH give you and on what port?

Answers

1. Home directory; `#` means you are root.
2. Least privilege — limited blast radius; root power borrowed per-command and auditable.
3. An encrypted remote shell (and file transfer); port 22.

3 Imaging and first boot

Flashing an image copies a pre-installed OS onto the card. **First-boot customisation** injects hostname/user/Wi-Fi/SSH into the boot partition so the Pi self-configures headless — which is why `raspi-config` later showed the hostname already filled in, and why your rebuild was cheap. The Windows “damaged drive” prompt is a **false alarm** (Windows can't read `ext4`); ignore it. **mDNS** resolves `dc-node-01.local` by local self-announcement — convenient, not dependable, hence the later static IP.

Skill Check

1. Why no monitor needed?
2. Windows says the card is damaged — do what?
3. How did `.local` resolve with no DNS?

Answers

1. First-boot customisation self-configured it.
2. Ignore it (ext4 misread; repair can corrupt).
3. mDNS/Avahi local name announcement.

4 Baseline capture

Read-only commands run *before* changes — the “before” photo. `hostnamectl` prints identity; its **Machine ID** is generated per install, so its later change proved your rebuild. `uname -a` = kernel details. `ip a` lists interfaces: `lo` (loopback), `eth0` (NO-CARRIER = unplugged), `wlan0` (Wi-Fi). The line `inet 10.0.0.249/24 ... dynamic ... valid_lft 172252sec` means a **DHCP** address (dynamic) on a finite lease — the problem §6 solves.

```

admin@dc-node-01: ~
up default qlen 1000
    link/ether 88:a2:9e:c2:77:d8 brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.249/24 brd 10.0.0.255 scope global dynamic noprefixroute wlan0
        valid_lft 172252sec preferred_lft 172252sec
    inet6 2607:fea8:bfa5:a100:8aa2:9eff:fec2:77d8/64 scope global dynamic mngtmp
addr proto kernel ra
    valid_lft 507676sec preferred_lft 98492sec
    inet6 fe80::8aa2:9eff:fec2:77d8/64 scope link proto kernel_ll
    valid_lft forever preferred_lft forever
admin@dc-node-01:~ $ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            3.9G   0  3.9G   0% /dev
tmpfs           1.6G  14M   1.6G   1% /run
/dev/mmcblk0p2   58G   6.8G   49G  13% /
tmpfs           4.0G  336K   4.0G   1% /dev/shm
tmpfs           5.0M   48K   5.0M   1% /run/lock
tmpfs           1.0M   0   1.0M   0% /run/credentials/systemd-journald.service
tmpfs           4.0G   16K   4.0G   1% /tmp
/dev/mmcblk0p1  505M   87M   418M  18% /boot/firmware
tmpfs           807M  256K   806M   1% /run/user/1000
tmpfs           1.0M   0   1.0M   0% /run/credentials/getty@tty1.service
tmpfs           1.0M   0   1.0M   0% /run/credentials/serial-getty@ttyAMA10.ser
vice
admin@dc-node-01:~ $

```

Figure 1: `df -h`: root on `/dev/mmcblk0p2`. `mmcblk` = SD card, so you were booting microSD, not SSD.

`cat /proc/device-tree/model` reads the board’s own identity from `/proc` (a virtual filesystem the kernel exposes) — authoritative.

Skill Check

1. Why baseline first?
2. Which two words prove DHCP not static?
3. How did `df -h` reveal an SD card?
4. What is `/proc`?

Answers

1. It’s the reference every later verification compares to.
2. `dynamic` + `valid_lft <sec>`; static shows neither (`forever`).
3. Device `mmcblk0p2`; `mmcblk` = SD/MMC.
4. A virtual filesystem exposing live kernel/hardware data.

5 Updating the system

`apt` is Debian's package manager. `apt update` refreshes the *catalog* (downloads nothing installable); `apt full-upgrade` installs newer versions and may remove packages to fix dependencies; `-y` auto-confirms. In `A && B`, **B runs only if A succeeded** — which is why your `supd` typo was harmless: the update half ran, the misspelled half failed alone. `rpi-eeeprom-update -a` updates bootloader firmware, verified only *after* a reboot.

Skill Check

1. `update` vs `full-upgrade`?
2. What does `&&` guarantee?
3. Why reboot before verifying firmware?

Answers

1. Catalog refresh vs installing newer versions (may remove packages).
2. Second command runs only if the first succeeded.
3. Firmware is staged and applied at boot, so “up to date” is only meaningful post-reboot.

6 Networking deep dive

The hardest section — it broke your build.

6.1 IP addresses and CIDR

`/24` = first 24 bits are the network, last 8 the host — so all `10.0.0.x` share a network. `/28` = only 16 addresses (14 usable), which is why a phone hotspot holds few devices. **The lockout:** you set `192.168.1.50/24` on a `10.0.0.0/24` node — different networks, so the Pi could reach nothing and dropped your session.

6.2 Gateway and DNS

Gateway (`10.0.0.1`) = the router, your exit to the internet. DNS (`8.8.8.8`) translates names to addresses. Your `8.4.4.4` typo was a broken *secondary* — invisible until the primary failed.

6.3 DHCP vs static; NetworkManager

DHCP = auto, leased, changes. Static = fixed by hand. Servers need a stable address. Control it with `nmcli`; the profile has a *name* you must **discover, never assume** (yours changed three times). `ipv4.method manual` is the switch to static; `connection up` deactivates and rebuilds the connection (which is why doing it over your own link can disconnect you).

6.4 APIPA

A flood of `169.254.x.x` means a host self-assigned because **no DHCP answered** — a whole-segment fault, not just the node.

Skill Check

1. Why did `192.168.1.50/24` on `10.0.0.0/24` disconnect the Pi?
2. `/24` vs `/28` host counts?
3. Tell DHCP from static in `ip a`?
4. `169.254.x.x` everywhere — what and what does it rule out?
5. Why always `nmcli connection show` first?

Answers

1. Different networks; no route to gateway/peer, so all traffic including SSH died. 2. /24=254 usable, /28=14 usable. 3. DHCP: `dynamic+finite valid_lft; static: forever, no dynamic`. 4. APIPA/link-local — no DHCP answered; rules out “only the node is broken.” 5. Profile names aren’t fixed (yours changed three times); assuming gives “unknown connection.”

7 Users and permissions

`/etc/passwd` = account metadata (world-readable, no passwords). `/etc/shadow` = password hashes (root-only). That is why `passwd redji` failed but `sudo passwd redji` worked — writing another user’s password touches root-only `shadow`. `usermod -aG docker admin: -G` sets supplementary groups; `-a` **appends** — without it you *replace* the whole list and can strip `sudo`. Group changes apply only in a new session.

Skill Check

1. Why is `/etc/shadow` root-only? 2. Why did `passwd redji` fail but `sudo passwd redji` work? 3. What does `-a` prevent? 4. Why did Docker still need `sudo` right after `usermod -aG`?

Answers

1. It holds password hashes; restricting it blocks offline cracking. 2. Writing another user’s password needs root-owned `shadow`. 3. Overwriting all supplementary groups (losing `sudo`). 4. Group membership applies only to new sessions.

8 SSH keys and hardening — the heart of the lab

8.1 Asymmetric crypto

A **keypair**: private key stays on your laptop; public key goes to the server. The server verifies a signature only the private key could produce, so **no secret ever crosses the network**. **Ed25519** is preferred over RSA — strong, tiny, fast.

8.2 Distributing the key

`scp` pushes the *public* key up. It must land in `~/.ssh/authorized_keys`. **» appends; > overwrites** — `>` would wipe existing keys (harmless for you only because the file was empty). Permissions matter: `sshd` silently ignores `authorized_keys` if `~/.ssh` is group/world-writable (`chmod 700 ~/.ssh; chmod 600 authorized_keys`).

8.3 Host keys

On first connect the *server* proves its identity with a fingerprint saved to `known_hosts`; a later change means a possible man-in-the-middle.

8.4 The include-ordering trap (D3)

Your drop-in `99-dc-hardening.conf` was overridden because `sshd_config` uses the **first** value read, and `cloud-init`’s `50-cloud-init.conf` sorted first with `PasswordAuthentication yes`. Renaming to `00-` fixed it. Always `sshd -t` (validate) before `systemctl restart ssh`.

8.5 Proving it: sshd -T and the negative test

sshd -T prints the *effective* config — the actual decision after merging every file. Then the **negative test**, the single most important proof in the lab:

```

admin@dc-node-01: ~
-rw-r--r-- 1 root root 102 Jul 22 18:23 99-dc-hardening.conf
/etc/ssh/sshd_config:57:#PasswordAuthentication yes
/etc/ssh/sshd_config:81:# PasswordAuthentication. Depending on your PAM configuration,
/etc/ssh/sshd_config:85:# PAM authentication, then enable this but set PasswordAuthentication
/etc/ssh/sshd_config.d/50-cloud-init.conf:1:PasswordAuthentication yes
/etc/ssh/sshd_config.d/99-dc-hardening.conf:2:PasswordAuthentication no
admin@dc-node-01:~ $ sudo mv /etc/ssh/sshd_config.d/99-dc-hardening.conf /etc/ssh/sshd_config.d/00-dc-hardening.conf
admin@dc-node-01:~ $ sudo sshd -t && sudo systemctl restart ssh && sudo sshd -T
| grep -E 'permitrootlogin|passwordauthentication|kbdinteractive'
permitrootlogin no
passwordauthentication no
kbdinteractiveauthentication no
admin@dc-node-01:~ $ sudo sshd -t && sudo systemctl restart ssh && sudo sshd -T
| grep -E 'permitrootlogin|passwordauthentication|kbdinteractive'
permitrootlogin no
passwordauthentication no
kbdinteractiveauthentication no
admin@dc-node-01:~ $ ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no admin@192.168.1.1
admin@192.168.1.1: Permission denied (publickey).
admin@dc-node-01:~ $

```

Figure 2: Negative test: forcing password-only auth and being *refused* — the proof passwords are disabled. A working key never showed this.

```

ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no admin@<ip>
# PASS = Permission denied (publickey).

```

The deepest lesson: configuration that is *present* is not configuration that is *effective*. A working key proves the key works; only the negative test proves passwords are *refused*. Test the refusal.

Skill Check

- How does key login prove identity without sending a secret?
- > vs »?
- Key ignored, no error — check what first?
- Why did 99- lose to 50-?
- Write the negative test and its “pass” output.
- Why isn’t a successful key login proof that passwords are off?

Answers

- The server verifies a signature only the private key could make; the key never leaves your machine.
- > overwrites, » appends.
- Permissions: ~/.ssh=700, authorized_keys=600.
- First value wins; drop-ins load alphabetically; 50-(yes) read before 99-(no).
- ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no admin@<ip> → Permission denied (publickey).
- Key and password auth are independent; only forcing password-only and being refused proves it.

9 Firewall, fail2ban, patching, time

ufw: `default deny incoming` (nothing in unless allowed), `allow OpenSSH before enable` (else you cut your own session), rules for both v4 and v6. **fail2ban:** bans IPs after repeated failures; `jail.conf`→`jail.local` survives upgrades; `systemctl enable -now` = start now + every boot; the journal backend reads systemd’s log, not a text file. **unattended-upgrades:** auto-applies security patches (answers the 60-pending-packages problem). **timedatectl:** `synchronized: yes` + `NTP active`; RTC in UTC is correct. Note: `NTP service: active` describes the daemon, not success — with DNS broken it can be “active” yet unsynchronised (D8).

```

admin@dc-node-01: ~
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp (OpenSSH) ALLOW IN Anywhere
22/tcp (OpenSSH (v6)) ALLOW IN Anywhere (v6)

admin@dc-node-01:~ $ sudo apt install fail2ban -y
Installing:
  fail2ban

Installing dependencies:
  python3-pyasyncore python3-pyinotify python3-systemd whois

Suggested packages:
  mailx system-log-daemon monit sqlite3 python-pyinotify-doc

Summary:
  Upgrading: 0, Installing: 5, Removing: 0, Not Upgrading: 128
  Download size: 612 kB
  Space needed: 3,316 kB / 54.1 GB available

Get:1 http://deb.debian.org/debian trixie/main arm64 python3-systemd arm64 235-1

```

Figure 3: `ufw status verbose`: default-deny inbound, OpenSSH allowed on v4 and v6.

Skill Check

1. Why allow OpenSSH before enable?
2. Why `jail.local` not `jail.conf`?
3. `enable -now`?
4. How can NTP be “active” but unsynced?

Answers

1. Enabling first applies default-deny with no SSH exception → lockout.
2. Upgrades overwrite `jail.conf`; `jail.local` persists.
3. Start now + on every boot.
4. “Active” = daemon running; broken DNS means it can’t reach servers, so it never synchronises.

10 Git, GitHub, and Docker

Git: add stages, commit snapshots, push uploads. `git init` made branch `master`; pushing `main` failed until `git branch -M main`. “Repository not found” meant the repo didn’t exist on GitHub yet (`remote add` creates nothing server-side). **Deleting a file later does not remove it from Git history** — keep secrets out of the first commit (D9). **Docker:** a container is an isolated app sharing the host kernel. `curl | sh` runs remote code as root (a supply-chain risk, mitigated by the GPG-signed repo it installs). `hello-world` pulled the `arm64` image — multi-arch working. The `docker` group is **root-equivalent** — a security decision, not a convenience.

Skill Check

1. Why did the first `git push origin main` fail? 2. Why “Repository not found” not “denied”? 3. Why keep secrets out of the first commit? 4. Why is `docker` group membership a security matter?

Answers

1. `init` made `master`; `main` didn’t exist — `git branch -M main`. 2. GitHub won’t confirm repos you can’t see; it truly didn’t exist yet. 3. History retains blobs recoverably; deletion later isn’t enough. 4. It’s root-equivalent (can mount host root in a container).

11 The nine defects as teaching cases

ID	What happened	Transferable lesson
D1	DNS secondary typo	A broken backup is invisible until the primary fails.
D2	Unexplained <code>sudo</code> account	Audit who can escalate.
D3	Hardening present but overridden	Present $\neq$ effective; test the negative.
D4	Docker needed <code>sudo</code>	A convenience fix can be a security decision.
D5	Key-file perms unverified	<code>sshd</code> silently ignores loose perms.
D6	Demo account had <code>sudo</code>	Lab artifacts accumulate privilege.
D7	Reboot unverified	Configured $\neq$ persistent.
D8	Clock unsynced after reboot	One low-level fault fakes many symptoms.
D9	Publishing private data	Git history is permanent; sanitise first.

Two meta-lessons: (1) *present $\neq$ effective* (D3, D8); (2) *fix the lowest layer first* (D8 — one stalled DHCP faked three symptoms).

12 Master skill-check exam

Skill Check

1. At the network level, why did 192.168.1.50/24 on 10.0.0.0/24 disconnect the Pi, and why no `nmcli` error?
2. What single fact proves the re-image was a genuine fresh install, and why can't it be faked?
3. Explain “first value wins” for 99- vs 00- drop-ins.
4. Write the negative SSH test and its “pass” output.
5. Why is a successful key login insufficient evidence of hardening?
6. 169.254.x.x everywhere — mechanism, and what it rules in/out?
7. Why must `ufw allow OpenSSH` precede `ufw enable`?
8. The rule shared by the SSH drop-in, `jail.local`, and temp-file editing?
9. Why a DHCP reservation over a host static for a node that moves?
10. Why keep sensitive screenshots out of the *first* commit?
11. NTP `active` but `NTPSynchronized no` — how both, and root cause?
12. State the two meta-lessons with one incident each.

Answers

1. Different subnets; no route to gateway/peer, SSH died. `nmcli` validated syntax, not reachability.
2. The Machine ID changed; it's generated on first boot of a fresh install, so a repair can't reproduce it.
3. `sshd_config` takes the first value read; drop-ins load alphabetically, so 00- is read before 50-/99- and wins.
4. `ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no admin@<ip> → Permission denied (publickey)`.
5. Key and password auth are independent; only the refusal proves passwords are off.
6. APIPA — no DHCP answered; rules in a segment/DHCP fault, rules out “only the node.”
7. Otherwise default-deny applies with no SSH exception and severs your session.
8. Put changes in an upgrade-safe override, validated before replacing the vendor file.
9. A host static breaks on subnet change; a MAC-keyed reservation is stable at home and still works elsewhere.
10. Git history is permanent and recoverable; deletion later doesn't remove it.
11. “Active” = daemon running; broken DNS (from a stalled DHCP activation) stopped it reaching servers. Root cause was D1's network, not NTP.
12. Present≠effective: D3 (config present, overridden). Fix lowest layer: D8 (one stalled DHCP → dead DNS, clock, SSH).

When the Master Exam is effortless, the lab is yours to Black Start — rebuild from zero, no manual, no internet. That is the BSL bar the timeline sets.